

OWASP Top 10

Injection

BIG school

BIGSEIO

GENERAMOS
NEGOCIO

Injection

Índice

- Introducción
- ¿Qué significa "Injection"?
- ¿Por qué ocurre esta vulnerabilidad?
- Ejemplos reales y típicos
- ¿Qué puede hacer un atacante si explota esto?
- ¿Cómo detectarlo?
- ¿Cómo prevenirlo?
- ¿Cómo mitigar si ya estás afectado?
- Checklist rápido
- Conclusiones



Injection

BIG school

Introducción

- Inyección: vulnerabilidad clásica y aún muy frecuente
- Datos externos interpretados como código o comandos
- Afecta BD, navegadores, SO y motores de plantillas
- Objetivo: comprender riesgos y aplicar defensas desde el diseño

¿Qué significa "Injection"?

- Ruptura entre datos (contenido) y código (instrucciones)
- El sistema ejecuta el input del usuario como lógica
- Validación en cliente no es suficiente
- Regla de oro: input siempre es dato, nunca instrucción

¿Por qué ocurre esta vulnerabilidad?

- Concatenación o interpolación insegura de inputs
- Falta de validación o saneamiento en servidor
- Confianza excesiva en controles del frontend
- Ausencia de escapado contextual al renderizar
- Uso de funciones peligrosas (eval, exec, innerHTML)

Ejemplos reales y típicos

- **SQLi:** consultas dinámicas sin parámetros
- **XSS:** scripts no escapados
(Reflejado/Almacenado/DOM)
- **Comandos:** input directo a shell o utilidades del SO
- **Template Injection:** motores que interpretan lógica externa
- **Patrones:** formularios sin validar, reflejo directo de input

¿Qué puede hacer un atacante si explota esto?

- Robo, modificación o eliminación de datos sensibles
 - Secuestro de sesiones o tokens (XSS)
 - Ejecución remota (RCE) o escalada de privilegios
 - Persistencia de malware o phishing desde el sitio
 - Incumplimiento normativo y daño reputacional

¿Cómo detectarlo?

- Desarrollo: code review de concatenaciones y eval
- Testing: SAST/DAST y pruebas con payloads estructurados
- Producción: logs de errores, tráfico con sintaxis inusual
- Señales: contenido extraño en UI, excepciones de parsing

¿Cómo prevenirlo?

- Validar y normalizar inputs en servidor (whitelist)
- Consultas parametrizadas o binding seguro (SIEMPRE)
- Escapado contextual según destino (HTML, JS, URL)
- Políticas estrictas: CSP, evitar eval, usar sanitizadores
- Mínimo privilegio en cuentas de BD y servicios

¿Cómo mitigar si ya estás afectado?

- Contener: WAF temporal o restringir endpoint
- Investigar: revisar logs y alcance del compromiso
- Corregir: aplicar parametrización y escapado en código
- Limpiar: eliminar payloads almacenados en BD/caché
- Rotar credenciales/tokens si hay evidencia de acceso indebido

Checklist rápido

- ✓ ¿Validación y normalización en servidor?
- ✓ ¿Consultas parametrizadas (sin concatenar)?
- ✓ ¿Escapado contextual según el contexto?
- ✓ ¿Evitado eval, exec, innerHTML con input externo?
- ✓ ¿Mínimo privilegio en BD/servicios?
- ✓ ¿CSP y headers de seguridad activos?
- ✓ ¿Tests automatizados para inputs inesperados?

Conclusiones

1. Persiste por confianza en el input y mala separación datos/código
2. Defensa clara: validar, parametrizar y escapar correctamente
3. Automatización (SAST/DAST) y CSP reducen riesgo drásticamente
4. Disciplina en desarrollo evita incidentes costosos
5. Regla final: datos son datos, nunca instrucciones



¡MUCHAS GRACIAS!

BIG school